

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

AI

Practical – VI

Roll No.: T001	Name: Soham Acharekar
Class: TYIT	Batch: 01
Date of Assignment: 19/8/26	Date/Time of Submission: 19/8/26

Probability Reasoning (Bayesian Networks & Inference)

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using Pgmpy.

CODE: Burglary-Alarm Network

pip install pgmpy

```
# Bayesian Network using pgmpy
# Example: Burglary-Alarm Network
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
# Step 2: Define Conditional Probability Tables
# P(Burglary)
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
# P(Earthquake)
cpd_earthquake = TabularCPD(
```

```

    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # JohnCalls = False
        [0.05, 0.90] # JohnCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30], # MaryCalls = False
        [0.01, 0.70] # MaryCalls = True
    ],

```

```

evidence=["Alarm"],
evidence_card=[2],
state_names={
    "MaryCalls": ["False", "True"],
    "Alarm": ["False", "True"]
}
)
# Step 3: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)
# Step 4: Check model correctness
print("Is Bayesian Network valid?", model.check_model())
# Step 5: Perform inference
inference = VariableElimination(model)
# Query: Probability of Burglary when John and Mary both call
result = inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)
print("\nProbability of Burglary given JohnCalls=True and MaryCalls=True:")
print(result)
print("Soham Acharekar T001")

```

OUTPUT:

```

Is Bayesian Network valid? True

Probability of Burglary given JohnCalls=True and MaryCalls=True:
+-----+-----+
| Burglary | phi(Burglary) |
+-----+-----+
| Burglary(False) | 0.7158 |
+-----+-----+
| Burglary(True) | 0.2842 |
+-----+-----+
Soham Acharekar T001

```

CODE: Rain-Sprinkler-Wet Grass Network (EXTRA)

```

# Bayesian Network using pgmpy
# Example: Rain-Sprinkler-Wet Grass Network
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD

```

```

from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Rain", "WetGrass"),
    ("Sprinkler", "WetGrass"),
    ("Rain", "Sprinkler")
])
# Step 2: Define Conditional Probability Tables
# P(Rain)
cpd_rain = TabularCPD(
    variable="Rain",
    variable_card=2,
    values=[[0.8], [0.2]],
    state_names={"Rain": ["False", "True"]}
)
# P(Sprinkler | Rain)
cpd_sprinkler = TabularCPD(
    variable="Sprinkler",
    variable_card=2,
    values=[
        [0.60, 0.99], # Sprinkler = False
        [0.40, 0.01] # Sprinkler = True
    ],
    evidence=["Rain"],
    evidence_card=[2],
    state_names={
        "Sprinkler": ["False", "True"],
        "Rain": ["False", "True"]
    }
)
# P(WetGrass | Rain, Sprinkler)
cpd_wetgrass = TabularCPD(
    variable="WetGrass",
    variable_card=2,
    values=[
        [1.00, 0.20, 0.10, 0.01], # WetGrass = False
        [0.00, 0.80, 0.90, 0.99] # WetGrass = True
    ],
    evidence=["Rain", "Sprinkler"],
    evidence_card=[2, 2],
    state_names={
        "WetGrass": ["False", "True"],
        "Rain": ["False", "True"],
        "Sprinkler": ["False", "True"]
    }
)
# Step 3: Add CPDs to model
model.add_cpds(
    cpd_rain,

```

```

    cpd_sprinkler,
    cpd_wetgrass
)
# Step 4: Check model correctness
print("Is Bayesian Network valid?", model.check_model())
# Step 5: Perform inference
inference = VariableElimination(model)
# Query: Probability of Rain when WetGrass is True
result = inference.query(
    variables=["Rain"],
    evidence={
        "WetGrass": "True"
    }
)
print("\nProbability of Rain given WetGrass=True:")
print(result)
print("Soham Acharekar T001")

```

OUTPUT:

```

Is Bayesian Network valid? True

Probability of Rain given WetGrass=True:
+-----+
| Rain      | phi(Rain) |
+-----+
| Rain(False) | 0.5869 |
+-----+
| Rain(True)  | 0.4131 |
+-----+
Soham Acharekar T001

```

2. Write a program to compute Posterior Probability using Bayesian Networks (Burglary|Alarm = True)

CODE:

```

# Posterior Probability using Bayesian Network
# Query: P(Burglary | Alarm = True)
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm")
])
# Step 2: Define probability of Burglary
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)

```

```

# Step 3: Define probability of Earthquake
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)

# Step 4: Define probability of Alarm
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)

# Step 5: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm
)

# Step 6: Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())

# Step 7: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)

print("\nPosterior Probability:")
print("P(Burglary | Alarm = True)")
print(result)
print("Soham Acharekar T001")

```

OUTPUT:

```

Is Bayesian Network valid? True

Posterior Probability:
P(Burglary | Alarm = True)
+-----+
| Burglary | phi(Burglary) |
+-----+
| Burglary(False) | 0.6264 |
+-----+
| Burglary(True) | 0.3736 |
+-----+
Soham Acharekar T001

```

CODE: (EXTRA)

```

# Posterior Probability using Bayesian Network
# Query: P(Disease | Fever = True)
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Disease", "Fever"),
    ("Disease", "Cough")
])
# Step 2: Define probability of Disease
cpd_disease = TabularCPD(
    variable="Disease",
    variable_card=2,
    values=[[0.90], [0.10]],
    state_names={"Disease": ["False", "True"]}
)
# Step 3: Define probability of Fever
# P(Fever | Disease)
cpd_fever = TabularCPD(
    variable="Fever",
    variable_card=2,
    values=[
        [0.95, 0.20], # Fever = False
        [0.05, 0.80] # Fever = True
    ],
    evidence=["Disease"],
    evidence_card=[2],
    state_names={
        "Fever": ["False", "True"],
        "Disease": ["False", "True"]
    }
)
# Step 4: Define probability of Cough

```

```

# P(Cough | Disease)
cpd_cough = TabularCPD(
    variable="Cough",
    variable_card=2,
    values=[
        [0.90, 0.30], # Cough = False
        [0.10, 0.70] # Cough = True
    ],
    evidence=["Disease"],
    evidence_card=[2],
    state_names={
        "Cough": ["False", "True"],
        "Disease": ["False", "True"]
    }
)
# Step 5: Add CPDs to model
model.add_cpds(
    cpd_disease,
    cpd_fever,
    cpd_cough
)
# Step 6: Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())
# Step 7: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Disease"],
    evidence={"Fever": "True"}
)
print("\nPosterior Probability:")
print("P(Disease | Fever = True)")
print(result)
print("Soham Acharekar T001")

```

OUTPUT:

```

Is Bayesian Network valid? True

Posterior Probability:
P(Disease | Fever = True)
+-----+
| Disease | phi(Disease) |
+-----+
| Disease(False) | 0.3600 |
+-----+
| Disease(True) | 0.6400 |
+-----+
Soham Acharekar T001

```